

SECURE IOT CHAT SERVER

Technical Architecture & Developer Reference

Version 1.0 | Open Source

1. Project Overview

This system is a real-time, TLS-encrypted messaging server built for IoT device-to-device communication. Devices authenticate with pre-shared tokens, join topic-based rooms, and exchange structured JSON messages at high throughput with built-in rate limiting.

Property	Value
Server Language	Python 3.x (asyncio)
Client Language	Python 3.x (asyncio)
Transport Protocol	TCP with TLS (mTLS optional)
Message Format	Newline-delimited JSON (NDJSON)
Default Port	9000
Max Message Size	4096 bytes (configurable)
Rate Limit	10 messages per 5-second window
Message Queue Size	200 per client

2. System Architecture

The server is structured as a layered set of independent, single-responsibility modules organized into four logical layers:

Layer	Module	Responsibility
Entry Point	main.py	Bootstraps and runs the async event loop
Configuration	config/settings.py	Reads environment variables and exposes typed constants

Security	security/tls.py	Builds the TLS SSL context from certificate files
Core Server	core/server.py	Accepts connections, routes messages, orchestrates all modules
Core	core/client.py	Represents a connected device with an async write queue
Core	core/auth_manager.py	Token-based authentication; returns device identity and room permissions
Core	core/room_manager.py	Tracks which clients are in which rooms
Core	core/broadcaster.py	Fans out a message to all room members except the sender
Core	core/rate_limiter.py	Sliding-window rate limiter per client connection
Core	core/logger.py	Centralized logging configuration
Protocol	protocol/parser.py	Decodes raw bytes to a Python dict via JSON
Protocol	protocol/validator.py	Ensures required fields are present before processing
Client SDK	device_client.py	Reusable client class for IoT devices to connect and communicate

3. Module Reference

3.1 Configuration — settings.py

All configuration is loaded from environment variables at startup. Sensible defaults are provided for local development.

Environment Variable	Default	Description
HOST	0.0.0.0	Bind address for the server
PORT	9000	TCP port the server listens on
MAX_MESSAGE_SIZE	4096	Maximum raw message length in bytes

QUEUE_SIZE	200	Maximum messages buffered per client send queue
RATE_LIMIT_MESSAGES	10	Allowed messages per rate-limit window
RATE_LIMIT_WINDOW	5	Rate-limit sliding window in seconds

3.2 TLS Security — `tls.py`

The server enforces TLS on every connection using Python's `ssl` module. Certificate files must be present before the server starts.

```
# Required certificate files
certs/server.crt    # PEM-encoded server certificate
certs/server.key    # PEM-encoded private key
```

The client SDK disables hostname verification and certificate validation by default, which is suitable for development and internal networks. For production deployments, set `check_hostname = True` and `verify_mode = ssl.CERT_REQUIRED`, and distribute a CA bundle to all devices.

3.3 Authentication — `auth_manager.py`

Authentication is token-based. Each token maps to a device identity and a list of rooms that device is permitted to join. The token store is currently an in-memory dictionary defined at class initialization.

Token	Device ID	Permitted Rooms
device_token_1	sensor_1	factory, alerts
device_token_2	sensor_2	factory
device_token_3	sensor_3	site
device_token_4	sensor_4	site

For production use, replace the in-memory dictionary with a persistent store such as a database or a secrets manager (e.g. HashiCorp Vault, AWS Secrets Manager). Tokens should be rotated regularly and revoked on device decommission.

3.4 Client Object — `client.py`

Each accepted TCP connection is wrapped in a Client instance. The client maintains:

- reader / writer: asyncio stream handles for the TCP connection

- `authenticated`: bool flag set to True after a successful connect handshake
- `device_id`: string identifier populated from the auth token
- `allowed_rooms`: list of rooms the device may join, from the token record
- `joined_rooms`: set of rooms the device has actively joined in this session
- `send_queue`: bounded `asyncio.Queue` with a configurable maximum size
- `writer_task`: background coroutine that drains the send queue in order

If the send queue is full when a broadcast arrives, the message is silently dropped for that client. This prevents a slow client from blocking the event loop or causing unbounded memory growth.

3.5 Room Management — `room_manager.py`

Rooms are created implicitly when the first client joins. The room registry is a `defaultdict(set)` keyed by room name. Available methods are:

Method	Description
<code>join(room, client)</code>	Adds the client to the room set and records the room in <code>client.joined_rooms</code>
<code>leave(room, client)</code>	Removes the client from the room set (called during disconnection cleanup)
<code>get_clients(room)</code>	Returns the current set of clients in a room (empty set if room does not exist)

3.6 Message Broadcasting — `broadcaster.py`

When a device publishes a message to a room, the broadcaster wraps the payload in a standard envelope and delivers it to every other client currently in that room.

The outgoing envelope always contains the following fields:

Field	Description
<code>type</code>	Always the string "message"
<code>room</code>	The room name to which the message was published
<code>from</code>	The <code>device_id</code> of the sending device
<code>timestamp</code>	Unix epoch seconds at the time of broadcast
<code>data</code>	The raw data payload from the publisher

3.7 Rate Limiting — `rate_limiter.py`

Each client gets its own `RateLimiter` instance using a sliding-window algorithm. A deque holds the timestamps of recent messages. On each call to `allow()`, timestamps older than the window are evicted. If the remaining count meets or exceeds the limit, the call returns `False` and the server disconnects the client.

Default configuration allows 10 messages per 5-second window. Both values are adjustable via environment variables.

3.8 Protocol — `parser.py` and `validator.py`

All messages on the wire are newline-delimited JSON. Parsing and validation are split into two dedicated classes:

- `MessageParser.parse(raw)`: Decodes bytes to a Python dict. Raises an exception on malformed JSON, which the server catches and treats as a fatal protocol error, disconnecting the client.
- `MessageValidator.validate(msg)`: Currently enforces that a `type` key is present. Any message without a `type` field is rejected.

The validator is intentionally minimal. Add field-level validation here (e.g. enforcing that message payloads include a room key, or that data values conform to expected schemas) as requirements grow.

4. Message Protocol

All communication between devices and the server uses newline-delimited JSON over the TLS TCP connection. Each JSON object must be terminated by a single newline character (`\n`).

4.1 Client-to-Server Messages

`connect`

Sent immediately after establishing a TCP connection to authenticate the device.

```
{ "type": "connect", "token": "device_token_1" }
```

`join`

Subscribes the device to a room. The room must be listed in the device's token permissions.

```
{ "type": "join", "room": "factory" }
```

`message`

Publishes a data payload to a room. The device must have already joined the room in the current session.

```
{ "type": "message", "room": "factory", "data": { "temperature": 34 } }
```

4.2 Server-to-Client Messages

connected

Sent by the server immediately after successful authentication.

```
{ "type": "connected", "device_id": "sensor_1" }
```

message (broadcast)

Received by all room members except the original sender.

```
{
  "type": "message",
  "room": "factory",
  "from": "sensor_1",
  "timestamp": 1715300000,
  "data": { "temperature": 34 }
}
```

5. Connection Lifecycle

Step	Action	Outcome on Failure
1	Client opens TLS TCP connection to server	Connection refused if cert invalid
2	Client sends connect message with token	Server disconnects if token unknown
3	Server responds with connected message	Session enters authenticated state
4	Client sends one or more join messages	Invalid room is silently ignored; auth violations are also ignored
5	Client sends message packets	Invalid room membership is silently ignored; rate limit exceeded closes connection
6	Client disconnects (clean or abrupt)	Server removes client from all rooms and discards client object

6. Device Client SDK

The DeviceClient class in device_client.py provides a high-level interface for any Python IoT device to connect to the server. The following methods are available:

Method	Description
<code>connect()</code>	Opens TLS TCP connection and sends the authentication handshake
<code>subscribe(room)</code>	Sends a join message to the server for the specified room
<code>publish(room, data)</code>	Sends a message packet with a dict payload to the specified room
<code>receive()</code>	Awaits and returns the next decoded JSON message from the server
<code>close()</code>	Closes the writer and waits for the connection to shut down cleanly

6.1 Sender Example

The sender.py script demonstrates a device that authenticates, joins a room, and continuously publishes sensor data:

```
client = DeviceClient('127.0.0.1', 9000, 'device_token_1')
await client.connect()
await client.subscribe('factory')
while True:
    await client.publish('factory', {'temperature': random.randint(20, 40)})
    await asyncio.sleep(1)
```

6.2 Receiver Example

The reciever.py script demonstrates a device that authenticates, joins a room, and listens for incoming messages:

```
client = DeviceClient('127.0.0.1', 9000, 'device_token_2')
await client.connect()
await client.subscribe('factory')
while True:
    msg = await client.receive()
    if msg is None: break
    print('[MESSAGE]', msg)
```

7. Error Handling and Disconnect Conditions

The server treats most errors as fatal at the protocol level, disconnecting the offending client immediately. This is a deliberate design decision to keep the server stateless and simple.

Condition	Server Action
Empty read (client closed connection)	Exit read loop, begin cleanup
Message exceeds MAX_MESSAGE_SIZE bytes	Disconnect client
Rate limit exceeded	Disconnect client
JSON parse error	Disconnect client
Message missing type field	Disconnect client
connect sent with unknown token	Disconnect client
Any message sent before connect	Disconnect client
join to a non-permitted room	Message silently ignored, session continues
message to a room not joined	Message silently ignored, session continues

8. Deployment & Configuration Guide

8.1 Certificate Generation (Development)

For local development, generate a self-signed certificate using OpenSSL:

```
mkdir -p certs
openssl req -x509 -newkey rsa:4096 -keyout certs/server.key \
  -out certs/server.crt -days 365 -nodes \
  -subj '/CN=localhost'
```

8.2 Environment Variables (.env)

```
HOST=0.0.0.0
PORT=9000
MAX_MESSAGE_SIZE=4096
QUEUE_SIZE=200
RATE_LIMIT_MESSAGES=10
RATE_LIMIT_WINDOW=5
```

8.3 Running the Server

```
pip install python-dotenv
python main.py
```


8.4 Running the Example Clients

```
# In one terminal:
python reciever.py

# In another terminal:
python sender.py
```

9. Extension Points

The following improvements are recommended for production deployments:

- Persistent token store: Replace the in-memory dict in AuthManager with a database query or a call to a secrets manager.
- Richer validation: Extend MessageValidator to enforce per-type field requirements (e.g. data must be a dict for message type).
- Client-side TLS verification: Enable check_hostname and CERT_REQUIRED in DeviceClient for production networks.
- Graceful shutdown: Notify connected clients before the server process terminates.
- Metrics and observability: Expose Prometheus metrics for connected clients, messages per second, and rate-limit events.
- Persistence: Optionally persist messages to a time-series database (e.g. InfluxDB, TimescaleDB) for historical queries.
- Mutual TLS (mTLS): Require client certificates for stronger device identity assurance.

End of Document | Secure IoT Chat Server | v1.0